

Data Structures & Algorithms Visual Diagrams

1. Array Operations

```

Array: [10, 20, 30, 40, 50]
Index:  0   1   2   3   4

Access:      O(1)
    arr[2] → 30

Insert at end:  O(1)
    [10, 20, 30, 40, 50, 60]

Insert at middle: O(n)
    [10, 20, 25, 30, 40, 50]
        ↑ Shift all elements →

Delete:      O(n)
    [10, 30, 40, 50]
        ↑ Shift elements ←

Search:      O(n)
    Linear scan through elements
  
```

2. Linked List Structure

```

Singly Linked List:

Head → [10|→] → [20|→] → [30|→] → [40|NULL]
      data next

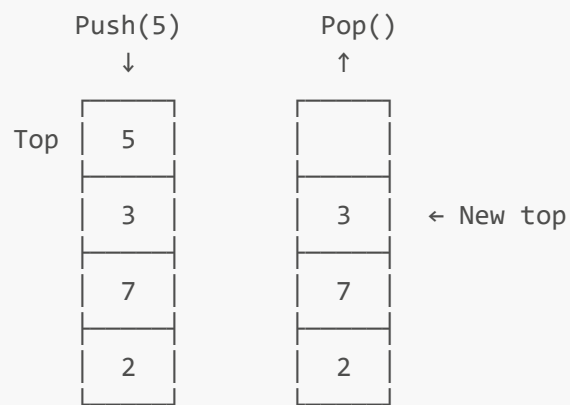
Operations:
- Insert at head: O(1)
- Insert at tail: O(n) without tail pointer, O(1) with tail
- Delete at head: O(1)
- Search: O(n)
- Access by index: O(n)

Doubly Linked List:

NULL ← [10|↔] ↔ [20|↔] ↔ [30|↔] ↔ [40|NULL]
      prev data next

Can traverse backwards!
  
```

3. Stack (LIFO)



Operations: $O(1)$

- Push: Add to top
- Pop: Remove from top
- Peek: View top

Use Cases:

- Function call stack
- Undo mechanism
- Expression evaluation
- Backtracking

4. Queue (FIFO)



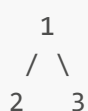
Operations: $O(1)$

- Enqueue: Add to rear
- Dequeue: Remove from front
- Peek: View front

Use Cases:

- BFS traversal
- Task scheduling
- Buffer management

5. Binary Tree Traversals



```

    / \
   4   5

```

Inorder (Left-Root-Right):

4 → 2 → 5 → 1 → 3

(For BST: gives sorted order)

Preorder (Root-Left-Right):

1 → 2 → 4 → 5 → 3

(Used for tree copying)

Postorder (Left-Right-Root):

4 → 5 → 2 → 3 → 1

(Used for tree deletion)

Level Order (BFS):

1 → 2 → 3 → 4 → 5

(Level by level)

6. Binary Search Tree (BST)

```

      50
     / \
    30  70
   / \  / \
  20 40 60 80

```

Properties:

- Left subtree < Root
- Right subtree > Root
- Inorder gives sorted sequence

Search 40:

50 → 30 → 40 ✓

(3 comparisons)

Best case: $O(\log n)$ - Balanced tree

Worst case: $O(n)$ - Skewed tree

```

    10
     \
    20
     \
    30 (Skewed - like linked list)

```

7. AVL Tree (Self-Balancing BST)

Before rotation:

After Right Rotation:



Balance Factor = height(left) - height(right)
Must be in {-1, 0, 1}

4 Rotation Types:

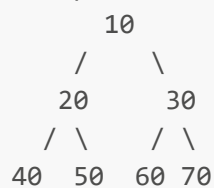
1. Left-Left (LL): Right Rotation
2. Right-Right (RR): Left Rotation
3. Left-Right (LR): Left then Right
4. Right-Left (RL): Right then Left

Always $O(\log n)$ operations!

8. Heap (Min Heap)

Array: [10, 20, 30, 40, 50, 60, 70]
Index: 0 1 2 3 4 5 6

Tree Representation:



Parent of i: $(i-1)/2$

Left child: $2i + 1$

Right child: $2i + 2$

Properties:

- Min Heap: Parent $\leq$ Children
- Max Heap: Parent $\geq$ Children

Operations:

- Insert: $O(\log n)$ - Bubble up
- Extract: $O(\log n)$ - Bubble down
- Peek: $O(1)$
- Heapify: $O(n)$

9. Hash Table

Hash Function: $h(\text{key}) = \text{key} \% 10$

Keys: [23, 43, 13, 27]

Index	Bucket
0	[]
1	[]
2	[]
3	[23 → 43 → 13] ← Collision (chaining)
4	[]
5	[]
6	[]
7	[27]
8	[]
9	[]

Collision Resolution:

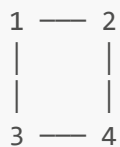
1. Chaining: Linked list at each bucket
2. Open Addressing: Find next empty slot
 - Linear Probing: $h(k)$, $h(k)+1$, $h(k)+2$, ...
 - Quadratic: $h(k)$, $h(k)+1^2$, $h(k)+2^2$, ...
 - Double Hashing: $h_1(k)$, $h_1(k)+h_2(k)$, ...

Average: $O(1)$

Worst: $O(n)$ - All keys in one bucket

10. Graph Representations

Graph:



Adjacency Matrix ($n \times n$):

	1	2	3	4
1	[0	1	1	0]
2	[1	0	0	1]
3	[1	0	0	1]
4	[0	1	1	0]

Space: $O(n^2)$

Edge check: $O(1)$

Adjacency List:

```

1 → [2, 3]
2 → [1, 4]
3 → [1, 4]
4 → [2, 3]

```

Space: $O(V + E)$

Edge check: $O(\text{degree})$

Use Matrix for dense graphs

Use List for sparse graphs

11. BFS vs DFS

Graph:



BFS (Queue - Level by level):

Visit: 1 → 2 → 3 → 4 → 5 → 6 → 7

Queue states:

[1]

[2, 3]

[3, 4, 5]

[4, 5, 6, 7]

...

Use: Shortest path, level order

DFS (Stack - Go deep first):

Visit: 1 → 2 → 4 → 5 → 3 → 6 → 7

Stack states:

[1]

[2, 3]

[4, 5, 3]

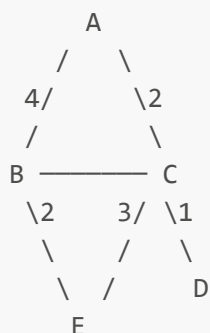
[5, 3]

...

Use: Cycle detection, topological sort

12. Dijkstra's Algorithm

Find shortest path from A to all nodes:



Step 1: Initialize

Distance: A=0, B= ∞ , C= ∞ , D= ∞ , E= ∞

Visited: []

Step 2: Visit A (distance 0)

Update: B=4, C=2

Visited: [A]

Step 3: Visit C (distance 2)

Update: D=3, E=5

Visited: [A, C]

Step 4: Visit D (distance 3)

Visited: [A, C, D]

Step 5: Visit B (distance 4)

Update: E= $\min(5, 4+2)=5$

Visited: [A, C, D, B]

Final distances: A=0, B=4, C=2, D=3, E=5

13. Binary Search

Array: [2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78]

Find: 23

Step 1: mid = $(0+10)/2 = 5$

arr[5] = 23 ✓ Found!

Find: 56

Step 1: mid = 5, arr[5]=23 < 56

Search right: [38, 45, 56, 67, 78]

Step 2: mid = 8, arr[8]=56 ✓ Found!

Comparison:

Linear: $O(n)$ - Check all elements

Binary: $O(\log n)$ - Eliminate half each time

Example with 1,000,000 elements:

Linear: ~500,000 comparisons (average)

Binary: ~20 comparisons ($\log_2 1,000,000 \approx 20$)

14. Merge Sort

[38, 27, 43, 3, 9, 82, 10]

Divide:

[38, 27, 43, 3] | [9, 82, 10]

```
[38, 27] [43, 3] | [9, 82] [10]
[38] [27] [43] [3] | [9] [82] [10]
```

Merge:

```
[27, 38] [3, 43] | [9, 82] [10]
[3, 27, 38, 43] | [9, 10, 82]
[3, 9, 10, 27, 38, 43, 82]
```

Time: $O(n \log n)$ always

Space: $O(n)$ - temporary arrays

Stable: Yes

15. Quick Sort

```
[10, 7, 8, 9, 1, 5]
```

Choose pivot: 5

Partition:

```
[1] < 5 | [5] | [10, 7, 8, 9] > 5
```

Recursively sort left and right:

```
[1] | [5] | [7, 8, 9, 10]
```

Final: [1, 5, 7, 8, 9, 10]

Partition Process (pivot = 5):

```
[10, 7, 8, 9, 1, 5]
  ↑i           ↑pivot
```

```
[1, 7, 8, 9, 10, 5] (swap 1 and 10)
  ↑i
```

```
[1, 5, 8, 9, 10, 7] (swap pivot to position)
  ↑final position
```

Time: $O(n \log n)$ average, $O(n^2)$ worst

Space: $O(\log n)$ - recursion stack

16. Dynamic Programming - Fibonacci

Recursive (Exponential $O(2^n)$):

```
      fib(5)
     /    \
  fib(4)  fib(3)
  /  \    /  \
fib(3) fib(2) fib(2) fib(1)
 /  \
fib(2) fib(1)
```

```
fib(2) fib(1)
```

Notice: fib(3), fib(2) computed multiple times!

Memoization (Top-Down DP):

```
memo = {}
def fib(n):
    if n in memo: return memo[n]
    if n <= 1: return n
    memo[n] = fib(n-1) + fib(n-2)
    return memo[n]
```

Time: $O(n)$, Space: $O(n)$

Tabulation (Bottom-Up DP):

```
dp = [0, 1]
for i in 2 to n:
    dp[i] = dp[i-1] + dp[i-2]
```

Time: $O(n)$, Space: $O(n)$

Space Optimized:

```
a, b = 0, 1
for i in 2 to n:
    a, b = b, a+b
```

Time: $O(n)$, Space: $O(1)$

17. Knapsack Problem (0/1)

Items: [(weight, value)]

Item 1: (2, 3)

Item 2: (3, 4)

Item 3: (4, 5)

Item 4: (5, 6)

Capacity: 8

DP Table:

Weight →	0	1	2	3	4	5	6	7	8
Item ↓									
0	0	0	0	0	0	0	0	0	0
1(2,3)	0	0	3	3	3	3	3	3	3
2(3,4)	0	0	3	4	4	7	7	7	7
3(4,5)	0	0	3	4	5	7	8	9	9
4(5,6)	0	0	3	4	5	7	8	9	10

Max value = 10 (items 2 and 4)

Recurrence:

```

dp[i][w] = max(
    dp[i-1][w],           // Don't take item i
    val[i] + dp[i-1][w-wt[i]] // Take item i
)

```

18. Longest Common Subsequence (LCS)

String 1: "ABCDGH"

String 2: "AEDFHR"

DP Table:

		A	E	D	F	H	R
		0	0	0	0	0	0
A		0	1	1	1	1	1
B		0	1	1	1	1	1
C		0	1	1	1	1	1
D		0	1	1	2	2	2
G		0	1	1	2	2	2
H		0	1	1	2	2	3

LCS Length = 3

LCS = "ADH"

Recurrence:

```

if s1[i] == s2[j]:
    dp[i][j] = 1 + dp[i-1][j-1]
else:
    dp[i][j] = max(dp[i-1][j], dp[i][j-1])

```

19. Two Pointers Technique

Two Sum (Sorted Array):

Target = 18

[2, 7, 11, 15, 23, 28]

↑ ↑
 left right

Step 1: 2 + 28 = 30 > 18 → right--

Step 2: 2 + 23 = 25 > 18 → right--

Step 3: 2 + 15 = 17 < 18 → left++

Step 4: 7 + 15 = 22 > 18 → right--

Step 5: 7 + 11 = 18 ✓ Found!

Time: O(n)

Space: O(1)

20. Sliding Window

Max Sum of K consecutive elements (K=3):

Array: [2, 1, 5, 1, 3, 2]

Window 1: [2, 1, 5] → sum = 8

Window 2: [1, 5, 1] → sum = 7

Window 3: [5, 1, 3] → sum = 9 ✓ Max

Window 4: [1, 3, 2] → sum = 6

Optimized:

Start: sum = 2+1+5 = 8

Slide:

sum = 8 - 2 + 1 = 7 (remove 2, add 1)

sum = 7 - 1 + 3 = 9 (remove 1, add 3)

sum = 9 - 5 + 2 = 6 (remove 5, add 2)

Time: O(n) instead of O(n×k)

21. Kadane's Algorithm (Max Subarray Sum)

Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

current_sum = 0

max_sum = -∞

Step by step:

i=0: -2 → current=max(0, -2)=-2, max=-2

i=1: 1 → current=max(-2+1, 1)=1, max=1

i=2: -3 → current=max(1-3, -3)=-2, max=1

i=3: 4 → current=max(-2+4, 4)=4, max=4

i=4: -1 → current=max(4-1, -1)=3, max=4

i=5: 2 → current=max(3+2, 2)=5, max=5

i=6: 1 → current=max(5+1, 1)=6, max=6

i=7: -5 → current=max(6-5, -5)=1, max=6

i=8: 4 → current=max(1+4, 4)=5, max=6

Max sum = 6

Subarray: [4, -1, 2, 1]

Time: O(n), Space: O(1)

22. Trie (Prefix Tree)

Insert: "cat", "car", "dog"



\$ = end of word marker

Search "car": root → c → a → r → \$ ✓

Search "ca": root → c → a (no \$) ✗

Operations: $O(m)$ where m = word length

Space: $O(\text{ALPHABET_SIZE} \times N \times M)$

Use Cases:

- Autocomplete
- Spell checker
- IP routing

23. Union-Find (Disjoint Set)

Find connected components:

Edges: (1,2), (2,3), (4,5)

Initial:

parent: [0, 1, 2, 3, 4, 5]

Each node is its own parent

After Union(1,2):

parent: [0, 1, 1, 3, 4, 5]

1 ← 2

After Union(2,3):

parent: [0, 1, 1, 1, 4, 5]

1 ← 2 ← 3

After Union(4,5):

parent: [0, 1, 1, 1, 4, 4]

1 ← 2 ← 3 4 ← 5

Components: {1,2,3}, {4,5}

Operations with Path Compression:

Find: $O(\alpha(n)) \approx O(1)$

Union: $O(\alpha(n)) \approx O(1)$

α = inverse Ackermann function

24. Topological Sort

DAG (Directed Acyclic Graph):

```
1 → 2 → 4
↓   ↓
3 → 5
```

In-degree (incoming edges):

1: 0, 2: 1, 3: 1, 4: 1, 5: 2

Kahn's Algorithm (BFS):

1. Find nodes with in-degree = 0: [1]
2. Process 1: output, remove edges
In-degree: 2:0, 3:0, 4:1, 5:2
3. Process 2, 3: output, remove edges
In-degree: 4:0, 5:0
4. Process 4, 5: output

Topological Order: 1 → 2 → 3 → 4 → 5

or: 1 → 3 → 2 → 5 → 4

Use Cases:

- Task scheduling with dependencies
- Build systems
- Course prerequisites

25. Floyd's Cycle Detection

Linked List with cycle:

```
1 → 2 → 3 → 4 → 5
      ↑       ↓
      8 ← 7 ← 6
```

Slow pointer (moves 1 step)

Fast pointer (moves 2 steps)

Step 1: Slow=1, Fast=1

Step 2: Slow=2, Fast=3

Step 3: Slow=3, Fast=5

Step 4: Slow=4, Fast=7

Step 5: Slow=5, Fast=3

Step 6: Slow=6, Fast=5

Step 7: Slow=7, Fast=7 ✓ Cycle detected!

Time: $O(n)$

Space: $O(1)$

Finding cycle start:

Reset slow to head

Move both at 1 step
They meet at cycle start (node 3)

This covers the major DSA concepts with visual diagrams!